MEERA

Personal AI Agent — Comprehensive Project Continuation Document

Project source of truth + current implementation state + continuation guide
October 2026

Purpose of this document. This document consolidates the original Personal AI Agent SRS, the architectural decisions made since it was created, the current Meera runtime/model experiments, and the exact point from which a future chat should continue. It is intended to prevent the project from being redefined or restarted in a new conversation.

1. Project Identity

Item	Current decision
Project / Agent name	Meera
Type	Personal AI agent, not merely a chatbot
Primary goals	Coding/reasoning + research/academic work + everyday productivity
Deployment	Local machine first
Initial cost model	Open-source/local/free-first; no paid AI API required for V1
Hardware constraint	Windows laptop with ~8 GB RAM and RTX 2050 4 GB VRAM
V1 interface	Open WebUI or comparable local web interface
Initial inference runtime	llama.cpp
Initial model family tested	Qwen3
Architecture principle	Stable compact core + replaceable edges

2. Product Vision

Meera is a modular, model-agnostic personal assistant that combines a local language model with persistent memory, preferences, skills, tools, planning, permissions, and future task workflows. The system should grow into a personal automation platform without requiring the core to be rebuilt whenever a new model, tool, protocol, or interface appears.

Core principle: Model = reasoning engine. Agent = model + memory + tools + planning + permissions + interfaces.

3. The Three-Layer Capability Principle

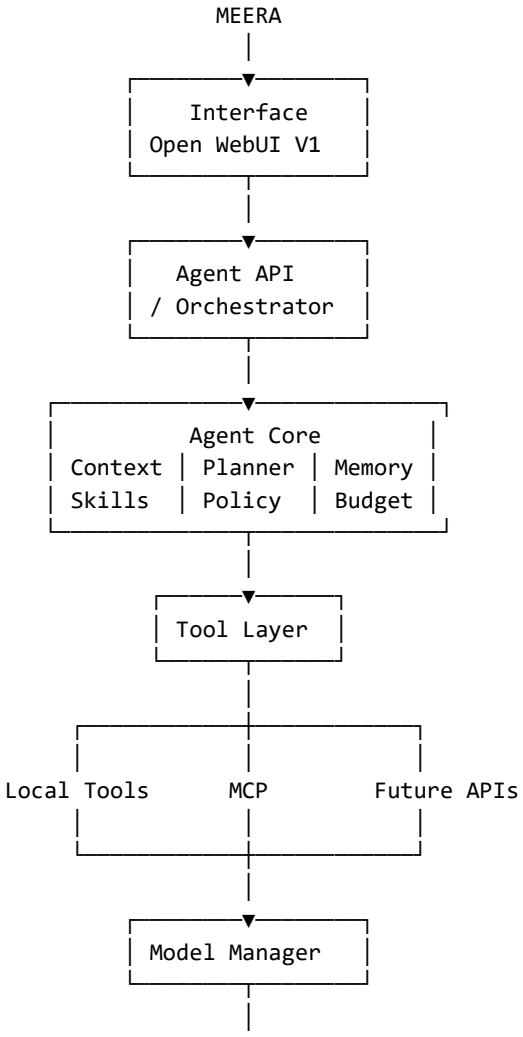
- **Skills decide what needs to happen.**
- **Tools decide how it happens.**
- **MCP is one way tools become available.**

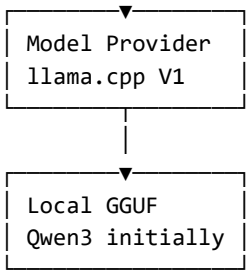
This is a locked architectural principle. Skills are the higher-level workflow/capability layer; tools are execution primitives; MCP is an integration mechanism/adapter rather than the brain or the entire internal architecture.

4. Design Philosophy

Principle	Meaning
Compact	Avoid unnecessary frameworks, services, abstractions, or dependencies.
Modular	Create replaceable boundaries where technology is likely to change.
Resource-conscious	Treat RAM/VRAM as first-class constraints; load expensive components only when needed.
Model-agnostic	The Agent Core must not depend on a particular model or inference runtime.
Tool-extensible	New capabilities should be addable without rewriting the core.
MCP-ready	Support MCP through the tool layer while retaining Meera's own tool contract.
Secure by architecture	The model does not receive authority simply because it can reason about an action.
Local-first	Prefer local processing for personal data where practical.
Measure before optimizing	Use actual measurements on the target machine instead of theoretical assumptions.
Stable core, replaceable edges	New models, runtimes, tools, memory stores, and interfaces should be adapters around a small core.

5. High-Level Architecture





6. Core Components and Responsibilities

Component	Responsibility	Keep it simple
Agent Core	Coordinates context, planning, memory, skills, permissions, tools and model calls.	Avoid a giant framework.
Model Manager	Load/unload models, select configuration, check resources, track lifecycle.	Own model lifecycle; do not embed business logic.
Model Provider	Uniform interface to inference backends.	llama.cpp first; Ollama/others can become adapters later.
Context Builder	Assemble current request, conversation context, relevant memory, skills and tool schemas.	Only include relevant context.
Planner	Break multi-step requests into useful steps/tool calls.	Start lightweight; introduce heavier orchestration only if needed.
Memory Manager	Handle conversation, long-term, preferences, project and task memory.	Do not persist every message.
Skill Registry	Discover and load capability/workflow definitions.	Skills describe how to accomplish goals.
Tool Registry	Register tool contracts and implementations.	Tools are execution primitives.
Permission / Policy Manager	Enforce READ/PREPARE/ACT and safety policies centrally.	Never let individual tools bypass it.
Resource Budget Manager	Track RAM/VRAM headroom and decide whether loading is safe.	Use measured thresholds.
Agent API	Expose the same backend to Open WebUI and future clients.	Keep UI-independent.

7. Skills, Tools and MCP

Skill = a capability/workflow. Tool = an executable primitive. MCP = one possible transport/integration source for tools.

```
Example:
Internship Skill
    ↓
Plan
    ↓
GitHub Tool ———————— local/MCP
Browser Tool ———————— MCP/future
File/Resume Tool —— local
Email Tool ———————— MCP/future
    ↓
Permission Manager
```

```
    ↓
User approval when required
    ↓
Execution
```

MCP should be treated as an expansion port. Adding an MCP server must not automatically grant unrestricted authority. Each exposed tool still enters Meera's tool registry and permission/policy boundary.

8. Guardrails and Safety Architecture

Locked principle: Intelligence does not equal authority.

The model must not be trusted to police itself. It may propose an action, but centralized enforcement determines whether that action can execute.

```
User
  ↓
Agent Core / Model
  ↓
Proposed action
  ↓
Policy + Permission Manager
  ├── DENY
  ├── ALLOW READ
  ├── PREPARE
  └── ACT → explicit approval when consequential
          ↓
        Tool Executor
          ↓
        Local Tool / MCP Tool
```

Level	Meaning	Examples
READ	Retrieve information; no external consequence.	Read file, inspect calendar, search repository.
PREPARE	Create a draft/preparation without committing externally.	Draft email, prepare application, generate resume.
ACT	Perform a consequential or externally visible action.	Send email, submit application, delete data, push changes.

V1 rule: ACT operations require explicit user confirmation. The permission system is centralized.

- MCP tools are not trusted by default.
- Web pages, emails, documents and tool outputs are treated as untrusted data, not instructions.
- Filesystem access should be restricted to approved locations.
- Terminal execution needs command/path/policy controls and eventually time/resource limits.
- Resource limits should constrain CPU/RAM/VRAM/processes/time where appropriate.
- A global stop/cancel mechanism should be available.
- Consequential actions should be auditable without logging sensitive content by default.
- Credentials should not be unnecessarily exposed to the model.

9. Memory Architecture

Memory scope	Purpose
Conversation	Current chat context and compact summaries.
Long-term	Selected useful facts that should survive across chats.

Preferences	Explicit response/behavior preferences and conventions.
Project	Facts scoped to a specific project.
Task	Persistent obligations, deadlines and task state.

Not every message becomes permanent memory. A memory policy should identify candidate memories, classify them, deduplicate them, and eventually let the user inspect/change stored preferences.

10. Model and Resource Management

The model is the dominant compute/resource consumer on the target machine. Meera therefore treats the model as an on-demand resource rather than a permanently resident process.

```
available RAM / VRAM
        ↓
system safety reserve
        ↓
agent overhead
        ↓
model budget
        ↓
safe to load?
   ├── no → smaller/different model or do not load
   └── yes → load
                ↓
           active usage
                ↓
         inactivity / stop
                ↓
             unload
```

Exact budget thresholds are not frozen yet. They must be established from real measurements on the user's normal workload, not from a clean-boot or theoretical baseline.

11. Actual Hardware Baseline

Resource	Observed value
OS	Windows 11 Home Single Language
Laptop	ASUS TUF Gaming F17 FX706HF
CPU	11th Gen Intel Core i5-11400H, 6 cores / 12 logical processors
System RAM	~7.73 GiB visible to Windows / ~7.9 GB physical
GPU	NVIDIA GeForce RTX 2050
VRAM	4096 MiB
NVIDIA driver	566.07
Driver-reported CUDA	12.7
Normal-use RAM baseline	~1.3 GiB available in earlier measurement
Post-model-exit RAM	1.73 GiB available in latest measurement
Storage policy	Use storage deliberately; free disk space is not a reason to consume it.

12. Runtime Setup Completed

Item	Status
Python	3.12.4
Git	2.47.1.windows.2
llama.cpp	b11496 / 0.6.0-dev build

llama.cpp package	Windows x64 CUDA 12 build
CUDA runtime DLLs	CUDA 12.4 DLL package added
CUDA backend	Verified
GPU discovery	llama.cpp reports CUDA0: NVIDIA GeForce RTX 2050
Project workspace	C:\Users\Arshit\Desktop\Overlord\Meera
Runtime path	Meera\runtime\llama.cpp
Model path	Meera\models

13. Model Benchmark Results

These are measured results from the actual laptop, not theoretical specifications.

Configuration	VRAM	RAM free	Working set	Private memory	Generation	Prompt
Qwen3-4B Q4_K_M, ctx 2048	~2829 MiB	~0.55 GiB	~1.84 GiB	~3.22 GiB	36.1 tok/s	145.6 tok/s
Qwen3-1.7B Q4_K_M, ctx 2048	~1415 MiB	~0.74 GiB	~1.24 GiB	~1.95 GiB	not captured	not captured
Qwen3-1.7B Q4_K_M, ctx 1024	~1301 MiB	~1.11 GiB	~1.25 GiB	~1.70 GiB	76.1 tok/s	254.6 tok/s

Interpretation: Qwen3-4B is usable for inference but leaves too little system RAM for a comfortable 8 GB Meera environment. Qwen3-1.7B is substantially lighter and faster. The 1024-context configuration is currently the strongest measured candidate, but context size 1536 should be tested once before finalizing the V1 profile.

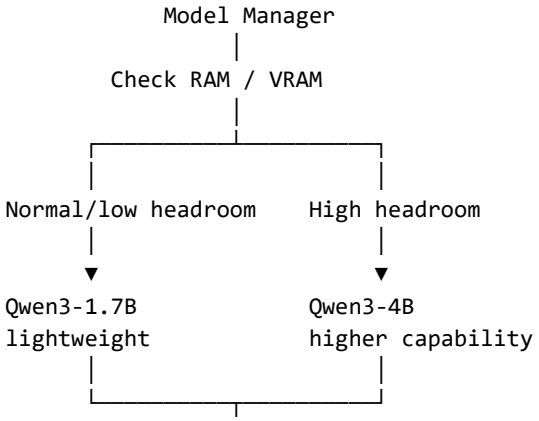
14. Current Model Decision

Current candidate for Meera V1 default: Qwen3-1.7B Q4_K_M.

Current measured configuration: context 1024, GPU offload enabled (-ngl 999), llama.cpp runtime. This is a candidate, not yet a permanently frozen setting.

Qwen3-4B Q4_K_M should be retained as a higher-capability/on-demand model for future resource-aware selection. The Model Manager should eventually choose model/configuration based on actual resource headroom.

15. Model Manager Concept



```
                ▼
           Agent Core
```

Later, additional local or remote models can be added without changing the Agent Core because inference is behind a ModelProvider interface.

16. ModelProvider Interface

```
class ModelProvider:
    def load(self): ...
    def unload(self): ...
    def generate(self, request): ...
    def stream(self, request): ...
    def health(self): ...
```

Initial implementation: llama.cpp provider. Future providers may include another local runtime or remote inference. Do not make the rest of Meera depend directly on llama.cpp.

17. Repository / Project Structure

```
Meera/
├── agent/              # Agent Core: orchestration, context, planner
├── model/              # ModelProvider + Model Manager
├── memory/             # Conversation, preferences, long-term/project/task memory
├── skills/             # Higher-level capabilities/workflows
├── tools/              # Tool contracts and local/MCP adapters
├── permissions/        # Policy and permission enforcement
├── api/                # Agent API
├── config/             # Configuration and resource policies
├── database/           # SQLite schema/migrations
├── tests/              # Unit/integration tests
├── ui/                 # Custom UI only if needed later
├── docs/               # Architecture/project documentation
│
├── models/             # Local GGUF models (do not commit to Git)
├── runtime/            # llama.cpp and future runtime binaries
├── cache/              # Temporary/cache data
└── logs/               # Runtime/audit logs
```

18. Initial Tool Contract

- name
- description
- input schema
- permission level
- execution handler
- result schema
- error handling

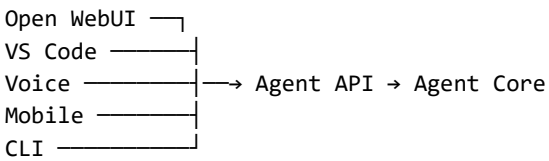
This contract should be simple enough for both local tools and MCP-backed tools. The Permission Manager must sit outside individual tool implementations so a tool cannot silently bypass policy.

19. Planned Tool Families

Tool family	Purpose	Priority
Files	Controlled read/write access to approved locations.	Early
Python	Controlled calculation/data analysis/code execution.	Early
Terminal	Controlled command execution with strong safety policy.	Early
Tasks	Persistent task/deadline operations.	Early
Email	Retrieve/search/summarize; later draft/send.	Later
Calendar	Read events, conflicts and deadlines.	Later
Browser/Web	Research and permitted web interaction.	Later
GitHub	Repositories/issues/PRs and future workflows.	Later
Internships	Search, filter, evaluate and prepare applications.	Later

20. Open WebUI and Future Interfaces

Open WebUI is an interface layer, not the core of Meera. The Agent API must remain the stable backend so future clients can be added.



Voice is an interface concern: microphone → speech-to-text → Agent API → Meera → text-to-speech → speaker. VS Code is postponed until the core/API is stable.

21. Technology Selection Rules

The ecosystem changes quickly. New open-source technologies should be evaluated when they solve a real problem, but novelty alone is not a reason to add a dependency.

- Prefer a mature/lightweight component if it already solves the problem.
- Prefer adapters over replacing the Agent Core.
- Benchmark resource cost on the actual laptop.
- Avoid duplicate frameworks that provide overlapping functionality.
- Use MCP where it meaningfully accelerates tool integration.
- Do not make MCP mandatory for every internal tool.
- LangChain/LangGraph are optional; introduce only if they materially simplify the implementation.
- Do not install databases/vector stores/services until a real requirement exists.
- Avoid permanently running expensive services.

22. Development Roadmap

Phase	Goal	Status
0 — Benchmark/Foundation	Environment, runtime, model	IN PROGRESS

	selection, RAM/VRAM/performance measurements.	
1 — Agent Core	Minimal local agent backend + model provider + conversation.	NEXT
2 — Interface	Connect Open WebUI to the Agent API.	Planned
3 — Memory	Conversation summaries, preferences, long-term/project/task memory.	Planned
4 — Skills + Core Tools	Skill registry + Files/Python/Terminal/Tasks.	Planned
5 — Planning + Permissions	Central policy, approval flow, tool execution safety.	Planned
6 — Productivity	Email, calendar, deadlines and task workflows.	Planned
7 — Internship	Opportunity discovery, JD matching, application preparation/tracking.	Planned
8 — Advanced Interfaces	Voice and VS Code clients.	Planned
9 — Model Expansion	Resource-aware routing, additional local/remote models.	Planned
10 — Hardening	Security, tests, observability, backup, packaging.	Planned

23. Explicit Non-Goals for V1

- Do not build a full custom ChatGPT-style UI before the core is stable.
- Do not keep a large model permanently resident.
- Do not require paid AI APIs.
- Do not start with VS Code integration.
- Do not give unrestricted terminal, filesystem, email or application-submission authority.
- Do not add dozens of tools before the tool contract and permission system are stable.
- Do not permanently lock Meera to Qwen3 or llama.cpp.
- Do not add large frameworks or infrastructure without a demonstrated need.

24. Current File/Workspace State

```
C:\Users\Arshit\Desktop\Overlord\Meera\
├── cache\
├── logs\
├── models\
│   ├── Qwen3-4B-Q4_K_M.gguf
│   └── Qwen3-1.7B-Q4_K_M.gguf
└── runtime\
    └── llama.cpp\
        ├── llama-cli.exe
        ├── llama-server.exe
        ├── ggml-cuda.dll
        ├── cudart64_12.dll
        ├── cublas64_12.dll
        └── ...
```

25. Immediate Next Steps

1. Finish Phase 0 model tuning with one controlled Qwen3-1.7B Q4_K_M test at context 1536.
2. Compare 1024 vs 1536 resource usage and generation speed.
3. Freeze an initial V1 model/resource profile based on measured results.
4. Define the minimal ModelProvider interface and llama.cpp provider.
5. Create the Meera Python project skeleton without unnecessary dependencies.
6. Implement a minimal Agent Core capable of receiving a message, building context, calling the model, and returning a response.
7. Add model lifecycle management: load on demand, unload after inactivity, and refuse unsafe loads.
8. Add the minimal API boundary.
9. Only then connect Open WebUI.

26. What the Next Chat Should NOT Do

- Do not restart product definition from scratch.
- Do not replace the current architecture merely because a new framework exists.
- Do not assume 4B is the default model; the measured resource profile currently favors 1.7B.
- Do not install a large collection of frameworks before a requirement exists.
- Do not bypass the centralized permission/guardrail design.
- Do not treat MCP as a substitute for the Agent Core.
- Do not forget the 8 GB RAM constraint.

27. Frozen Decisions / Source of Truth

- The project is named Meera.
- Meera is a personal AI agent, not just a chatbot.
- Primary use: coding/reasoning + productivity/research/academic assistance.
- 8 GB RAM is a first-class constraint.
- Local/open-source/free-first is the V1 direction.
- Model, Agent Core, memory, tools, skills, permissions and interfaces are decoupled.
- Open WebUI is the preferred V1 interface.
- llama.cpp is the selected initial inference runtime after GPU verification.
- Qwen3 is the initial model family; model choice remains replaceable.
- Qwen3-1.7B Q4_K_M is the current V1 default candidate; final context size still needs 1536 testing.
- Qwen3-4B Q4_K_M remains available as a higher-resource option.
- Skills decide what needs to happen.
- Tools decide how it happens.
- MCP is one way tools become available.
- MCP does not bypass Meera's tool registry or permission system.
- The model does not have inherent authority.
- READ / PREPARE / ACT permission levels are retained.
- Consequential ACT operations require explicit user confirmation.
- Tool outputs/web pages/documents are untrusted data, not instructions.
- Model lifecycle and resource safety are responsibilities of the Model Manager/Resource Budget Manager.
- Future interfaces must use the same Agent API/core.

28. Handoff Prompt for the Next Chat

Copy/paste the following into a new chat if needed:

```
You are continuing the Meera project. Use the attached/linked Meera Project Continuation Document
as the project source of truth.

Meera is a compact, modular, resource-conscious personal AI agent. Do not restart the product
definition.

Locked principles:
- Stable compact core + replaceable edges.
- Skills decide what needs to happen.
- Tools decide how it happens.
- MCP is one way tools become available.
- Model != authority.
- Centralized guardrails/permissions enforce READ/PREPARE/ACT.
- Consequential ACT operations require explicit user approval.
- Tool outputs/web pages/documents are untrusted data, not instructions.
- 8 GB RAM is a first-class constraint.
- Local/open-source/free-first.
- Model/runtime/interface/tool/memory separation is mandatory.

Current environment:
- Windows 11
- ASUS TUF Gaming F17 FX706HF
- i5-11400H, 6C/12T
- 7.73 GiB visible system RAM
- RTX 2050, 4096 MiB VRAM
- NVIDIA driver 566.07
- CUDA driver reports 12.7
- Python 3.12.4
- Git 2.47.1.windows.2
- llama.cpp b11496 / CUDA 12 build verified
- llama.cpp detects CUDA0: NVIDIA GeForce RTX 2050

Current models:
- Qwen3-4B-Q4_K_M.gguf
- Qwen3-1.7B-Q4_K_M.gguf

Measured results:
- Qwen3-4B Q4_K_M, ctx 2048: ~2829 MiB VRAM, ~0.55 GiB system RAM free, 36.1 tok/s generation,
145.6 tok/s prompt.
- Qwen3-1.7B Q4_K_M, ctx 2048: ~1415 MiB VRAM, ~0.74 GiB RAM free.
- Qwen3-1.7B Q4_K_M, ctx 1024: ~1301 MiB VRAM, ~1.11 GiB RAM free, 76.1 tok/s generation, 254.6
tok/s prompt.

Current candidate:
- Qwen3-1.7B Q4_K_M at ctx 1024.
- Test ctx 1536 once, then freeze the initial V1 model profile.
- Keep 4B as a higher-capability/on-demand option.

Immediate next task:
1. Run the 1.7B Q4_K_M benchmark at context 1536 under the same conditions.
2. Compare 1024 vs 1536.
3. Freeze initial model/resource configuration.
4. Begin Phase 1: minimal Python Agent Core + ModelProvider + llama.cpp provider +
```

```
lifecycle/resource manager.
5. Do not add unnecessary frameworks yet.
```

29. Source and Provenance

Primary architectural source: Personal_AI_Agent_SRS.docx (Project Foundation / SRS, Version 1.0, October 2026). This continuation document also records implementation decisions and measurements made after the original SRS was created.

Where this document records measured runtime values, those values come from the user's actual Windows/llama.cpp sessions during this project. Where it records architecture, it preserves the original SRS and subsequent project decisions.